

Activity#3

CODE:

main.py

```
1  from ArrayStack import ArrayStack as Stack
2
3  # Part 1: Balanced Expression Checker
4  1 usage new *
5  def is_balanced(expression):
6      stack = Stack()
7      matching_pairs = {'(': ')', '[': ']', '{': '}'
8
9      for char in expression:
10         if char in '([{':
11             stack.push(char) # Push opening bracket to the stack
12         elif char in ')]}':
13             if stack.is_empty() or stack.pop() != matching_pairs[char]:
14                 return False # unbalanced if stack top is not a closing bracket
15
16     return stack.is_empty() # balanced if stack is empty at the end
17
18  1 usage new *
19  def test_balanced_expressions():
20      expressions = [
21          "( ) ( ( ) ) { ( [ ( ) ] ) }", # Correct
22          "(( ( ) ( ( ) ) { ( [ ( ) ] ) } ) )", # Correct
23          ") ( ( ) ) { ( [ ( ) ] ) }", # Incorrect
24          "{ [ ] ) }", # Incorrect
25          "(" # Incorrect
26      ]
27
28      print("Balanced Expression Checker Results:")
29      for exp in expressions:
30         result = is_balanced(exp)
31         print(f"Expression: {exp} - Balanced: {result}")
32     print("\n") # space for readability
33
34  # Part 2: Reversing Lines in a File
35  1 usage new *
36  def reverse_lines_in_file(input_file, output_file):
37      stack = Stack()
```

```

37
38     # Try to open the input file
39     try:
40         with open(input_file, 'r') as file:
41             for line in file:
42                 stack.push(line)
43
44     except FileNotFoundError:
45         print(f"Error: The file {input_file} was not found.")
46         return
47
48     # Open the output file in write mode
49     with open(output_file, 'w') as file:
50         while not stack.is_empty():
51             reversed_line = stack.pop()
52             file.write(reversed_line)
53
54     print("Reversed lines written to {output_file} \n")
55
56
57     1 usage new *
58     def reverse_file_example():
59         input_file = 'myfile.txt'
60         output_file = 'reversed_output.txt'
61
62         # Reverses the lines in the file
63         reverse_lines_in_file(input_file, output_file)
64
65     # Run balanced checker and the file reverser
66     if __name__ == "__main__":
67         print("Part 1: Balanced Expression Checker")
68         test_balanced_expressions()
69
70         print("Part 2: Reverse File Lines")
71         reverse_file_example()

```

ArrayStack.py

```
1 usage new *
1 class ArrayStack:
    new *
2     def __init__(self):
3         self.stack = []
4
5     5 usages new *
5     def is_empty(self):
6         return len(self.stack) == 0
7
8     2 usages new *
8     def push(self, item):
9         self.stack.append(item)
10
11     2 usages new *
11     def pop(self):
12         if not self.is_empty():
13              return self.stack.pop()
14         return None
15
16     new *
16     def peek(self):
17         if not self.is_empty():
18             return self.stack[-1]
19         return None
20
21     new *
21     def size(self):
22         return len(self.stack)
23
```

myfile.txt

```
1 |this is the first line
2 |this is the second line
3 |this is the third line
4 |this is the fourth line
5 |this is the fifth line
6 |
```

OUTPUT:

```
Part 1: Balanced Expression Checker
Balanced Expression Checker Results:
Expression: ( )(( )){([ ( ) ])} - Balanced: True
Expression: ((( )(( )){([ ( ) ])})) - Balanced: True
Expression: )(( )){([ ( ) ])} - Balanced: False
Expression: ({[ ]}) - Balanced: False
Expression: ( - Balanced: False

Part 2: Reverse File Lines
Reversed lines written to {output_file}
```

reversed_output.txt

	main.py	 reversed_output.txt 
1	this is the fifth line	
2	this is the fourth line	
3	this is the third line	
4	this is the second line	
5	this is the first line	